

AuraCinema - Documentatie proiect

1. Scopul aplicatiei

AuraCinema este o aplicatie web full-stack care permite cautarea unui film dupa titlu, afiseaza informatii preluate din OMDb si propune o recomandare bazata pe scorul publicului. Aplicatia afiseaza titlul, anul, ratingul de varsta, durata, sinopsisul, posterul si sursa scorului.

Regulile de recomandare sunt:

- daca scorul publicului este peste 80%, utilizatorul primeste recomandarea de a viziona filmul imediat;
- daca scorul este sub 50%, utilizatorul primeste recomandarea de a evita filmul;
- intre 50% si 80%, aplicatia ofera o recomandare neutra;
- daca nu exista scor disponibil, aplicatia afiseaza datele filmului fara o recomandare ferma.

2. Tehnologii utilizate

- Frontend: React, JavaScript, Vite, CSS standard.
- Backend: Node.js, Express, CORS, dotenv.
- Stocare: MongoDB Atlas prin MongoDB Node.js Driver.
- API extern: OMDb API.
- Documentatie: Markdown, Mermaid si PDF generat local.

3. Arhitectura de ansamblu

Frontend-ul trimite o cerere catre backend pentru titlul cautat. Backend-ul verifica intai cache-ul din MongoDB Atlas. Daca intrarea exista si nu a expirat, raspunsul este intors direct. Daca intrarea lipseste, este expirata sau utilizatorul cere refresh, backend-ul interogheaza OMDb, normalizeaza raspunsul, salveaza intrarea in MongoDB si intoarce datele catre frontend.

flowchart LR

U[Utilizator] --> F[Frontend React]

F -->|GET /api/movies?title=...| B[Backend Express]

B --> C[(MongoDB Atlas)]

B -->|daca lipseste sau expira| O[OMDb API]

O --> B

C --> B

B --> F

F --> U

4. Use Case

Actorul principal este utilizatorul aplicatiei. Sistemul este AuraCinema.

```
flowchart LR
    User((Utilizator))
    subgraph AuraCinema
        UC1([Cauta film dupa titlu])
        UC2([Vizualizeaza detalii film])
        UC3([Primeste recomandare])
        UC4([Reimprospateaza datele])
    end
    User --> UC1
    User --> UC2
    User --> UC3
    User --> UC4
```

5. Diagrama de activitate

```
flowchart TD
    A([Start]) --> B[Utilizatorul introduce titlul]
    B --> C{Titlul este valid?}
    C -->|Nu| D[Afiseaza eroare de validare]
    C -->|Da| E{Exista in cache si nu este refresh?}
    E -->|Da| F[Returneaza datele din cache]
    E -->|Nu| G[Apeleaza OMDb API]
    G --> H{Filmul exista?}
    H -->|Nu| I[Afiseaza mesaj 404]
    H -->|Da| J[Normalizeaza datele si calculeaza recomandarea]
    J --> K[Salveaza in cache cu expirare]
    F --> L[Afiseaza poster, rating si sinopsis]
    K --> L
    L --> M([Stop])
```

6. Diagrama de interactiune

sequenceDiagram

participant U as Utilizator

participant F as Frontend React

participant B as Backend Express

participant C as MongoDB Atlas

participant O as OMDb API

U->>F: Introduce titlul filmului

F->>B: GET /api/movies?title=...

B->>C: Verifica documentul cache

alt Cache valid

C-->>B: Date film

else Cache lipsa/expirat

B->>O: Cerere dupa titlu

O-->>B: Date OMDb

B->>C: Upsert document cache

end

B-->>F: JSON cu film si recomandare

F-->>U: Afiseaza rezultatul

7. API si flux de date

Endpoint-uri:

- 'GET /api/health' - verifica daca serverul ruleaza.
- 'GET /api/movies?title=Movie+Name' - cauta filmul, folosind cache-ul daca este valid.
- 'GET /api/movies?title=Movie+Name&refresh=true' - ignora cache-ul si cere date noi din OMDb.

Raspunsul principal contine:

```
{
```

AuraCinema

Documentatie proiect - pagina 4

```
"title": "Guardians of the Galaxy",
"year": "2014",
"rated": "PG-13",
"runtime": "121 min",
"plot": "A group of intergalactic criminals...",
"poster": "https://...",
"scorePercent": 92,
"scoreSource": "Rotten Tomatoes",
"recommendation": "Ar trebui sa vizionati acest film chiar acum!",
"cached": true,
"cacheExpiresAt": "2026-05-06T12:00:00.000Z"
}
```

8. Cache si expirare

Cache-ul este salvat in MongoDB Atlas, in baza de date 'auracinema' si colectia 'movieCache'. Cheia documentului este titlul normalizat al filmului: fara spatii inutile si cu litere mici. Durata implicita a cache-ului este de 24 de ore si poate fi schimbata prin 'CACHE_TTL_HOURS'.

Backend-ul creeaza un index TTL pe campul 'expiresAt', astfel incat MongoDB poate sterge automat documentele expirate. Daca utilizatorul apasa reimprospatare, frontend-ul trimite 'refresh=true', iar backend-ul ocoleste cache-ul pentru cautarea respectiva.

9. Rulare locala

npm install

Terminal 1:

```
cd backend
npm run dev
```

Terminal 2:

```
cd frontend
npm run dev
```

In 'backend/.env' trebuie completate variabilele 'PORT', 'CORS_ORIGIN', 'CACHE_TTL_HOURS', 'MONGODB_URI', 'MONGODB_DB_NAME', 'MONGODB_CACHE_COLLECTION' si 'OMDB_API_KEY'. In 'frontend/.env' se pastreaza doar 'VITE_API_BASE_URL=http://localhost:4000'. Backend-ul ruleaza pe 'http://localhost:4000', iar frontend-ul pe 'http://localhost:5173'.